

Homework 1

Guangyu Qiu

2026-09-18

At first glance, we can see that the ball will end up in the hands of the last person who first touched it. Since everyone is sitting around a round table, my guess is that the person sitting opposite the host is the most likely to end up holding the ball (given $p = 0.5$).

Question 1

AI disclosure: this code was mostly hand-coded by me. The only help I received from AI # was that, at first, I used 'j' instead of 'i' inside the loop and did not set 'i <- j' # after the loop ended. I have now revised the code to use 'i' consistently throughout.

```
# say p is probability for heads, which mean left
present_party <- function(chair = 10, p = 0.5){
  x <- rep(0, chair)
  i <- 1 # the host hold the ball at the beginning
  x[i] <- 1
  n <- 0
  while (sum(x) != chair){
    a <- 2 * rbinom(1, 1, p) - 1 # if head, pass to left, then 1, otherwise -1
    i <- i + a
    if (i == 0) i <- chair else if (i == chair + 1) i <- 1
    x[i] <- 1
    n <- n + 1
  }
  return(c(i,n))
}

present_party()
```

```
## [1] 6 65
```

Question 2

```
# To prove it is correct, we should consider some extreme cases:
present_party(chair = 10, p = 1) # would return 10 and 9 rounds
```

```
## [1] 10 9
```

```
present_party(chair = 10, p = 0) # would return 2 and 9 rounds
```

```
## [1] 2 9
```

```
present_party(chair = 1, p = 0.5) # would return 1 and 0 rounds
```

```
## [1] 1 0
```

Question 3

```
set.seed(648)
sim <- rep(0,10000)
pass <- rep(0,10000)
for (j in seq_along(sim)){
  sim[j] <- present_party()[1]
  pass[j] <- present_party()[2]
}
head(sim,10)
```

```
## [1] 6 2 2 7 3 4 4 3 3 9
```

```
head(pass,10)
```

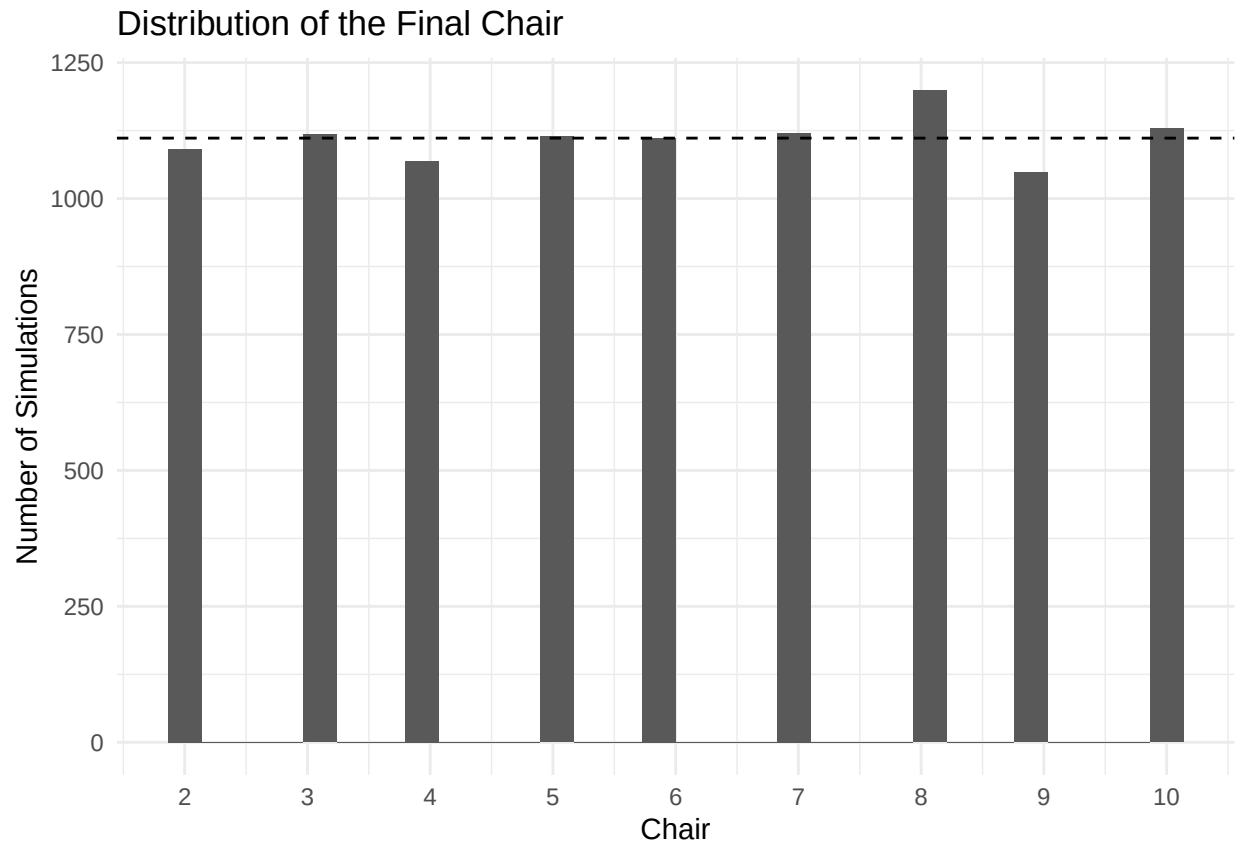
```
## [1] 36 56 69 81 53 47 55 30 17 102
```

Question 4

```
# AI disclosure: This code was created with the help of AI.
```

```
ggplot(data.frame(x = sim), aes(x = sim)) +
  geom_histogram() +
  scale_x_continuous(breaks = 2:10) +
  geom_hline(
    yintercept = 10000 / 9,
    linetype = "dashed")+
  labs(
    title = "Distribution of the Final Chair",
    x = "Chair",
    y = "Number of Simulations"
  ) +
  theme_minimal()
```

```
## `stat_bin()` using `bins = 30`. Pick better value `binwidth`.
```



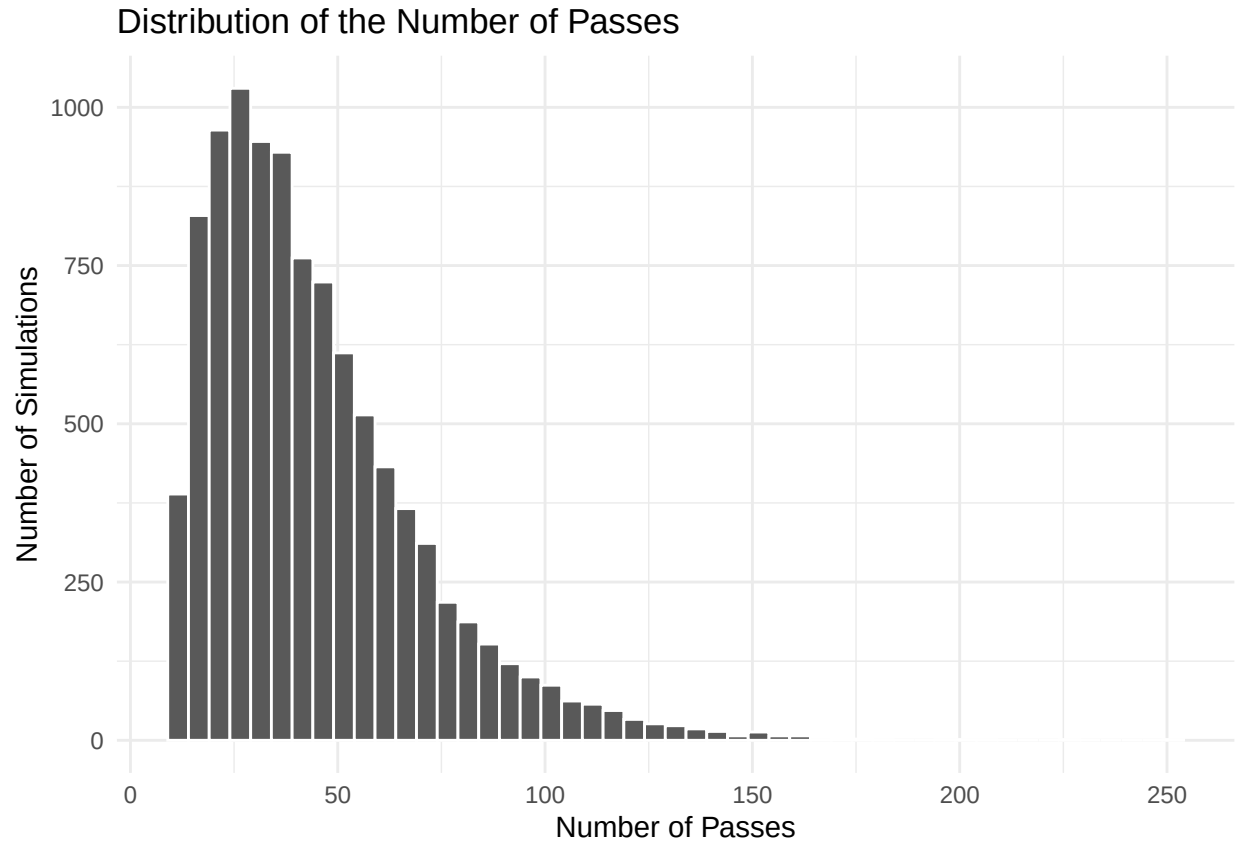
Question 5

```
summary(pass)
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##      9.0   26.0   39.0   44.9   58.0   250.0
```

AI disclosure: This code was created with the help of AI.

```
ggplot(data.frame(x = pass), aes(x = pass)) +
  geom_histogram(
    binwidth = 5,
    boundary = 9,
    color = "white"
  ) +
  labs(
    title = "Distribution of the Number of Passes",
    x = "Number of Passes",
    y = "Number of Simulations"
  ) +
  theme_minimal()
```



We find that the number of passes ranges from a minimum of 9 (which is reasonable) to a maximum of 250, with a mean of 44.9 and a median of 39. Since the mean exceeds the median, the distribution is right-skewed. The plot also reveals that as the number of passes increases, the count rises rapidly before gradually declining.

Question 6

First, the histogram appears to follow a uniform distribution, meaning that, except for the host, all chairs have the same probability of being the last one visited. I think the intuition is that the chairs that have already been visited always form one connected block, and the two boundaries of this block expand until they cover the whole circle. Since the direction of expansion is random, no chair is naturally more likely to be the last remaining gap.

Question 7

```
# AI disclosure: I originally planned to use a KS test, but ChatGPT pointed out that the
# KS test is designed for continuous distributions, whereas the outcome here is discrete.
chisq.test(table(sim), p = rep(1/9, 9))
```

```
##
## Chi-squared test for given probabilities
##
```

```
## data: table(sim)
## X-squared = 12.923, df = 8, p-value = 0.1145
```

The chi-squared goodness-of-fit test gave a p-value of 0.11, so I failed to reject the null hypothesis that all nine non-host chairs are equally likely to be the last chair visited.

Question 8 for loop part

```
# AI disclosure:
# I wrote the pseudocode for the for loop and asked ChatGPT to convert it into
# executable R code, then I checked and revised a little bit.

# The pseudocode was as follows:

# For-loop version:
# say prob = (0.1, 0.2, ..., 0.9), chair = n (I can decide it, say 11),
# for p in prob:
#   for i in 10000:
#     store [1] from present_party(n, prob[i]) in a matrix
#   within the first loop, calculate the mean after completing 10000 simulations.

# My definition of "unfair" is:
# Calculate the probability of each chair as count / total, then subtract the smallest
# probability from the largest probability. This gives the probability difference.
# When p = 0.5, ideally every chair should have a probability of 0.1, so the probability
# difference should be 0.

# Therefore, plot1 has p from 0.1 to 0.9 (length(prob)) on the X-axis and probability
# difference (unfairness) on the Y-axis. table1 contains the same information.

# Plot2 has the n-1 non-host chairs (chairs 2 through n) on the X-axis and the mean count
# for each chair on the Y-axis. There are length(prob) lines in total, one for each
# scenario, with 10000 simulations for each scenario.

# For loop version -----
set.seed(648)
prob <- seq(0.1, 0.9, by = 0.1)
chair <- 11
result <- list()

for (p in prob) {

  final_chair <- numeric(10000)

  for (i in 1:10000) {
    simulation <- present_party(chair, p = p)
    final_chair[i] <- simulation[1] # final holder
  }

  # probability that each chair is the final holder
```

```

counts <- table(factor(final_chair, levels = 1:chair))
probabilities <- as.numeric(counts) / length(final_chair)
# for this inner loop under given p, calculate unfairness
unfairness <- max(probabilities) - min(probabilities)

result[[as.character(p)]] <- list(
  probabilities = probabilities,
  unfairness = unfairness
)
}

# Table 1: p and unfairness
table1 <- data.frame(
  `Prob of Head` = paste0(prob * 100, "%"),
  unfairness = sapply(result, function(x) x$unfairness),
  check.names = FALSE
)

print(table1, row.names = FALSE)

```

```

## Prob of Head unfairness
##      10%      0.8867
##      20%      0.7557
##      30%      0.5774
##      40%      0.3342
##      50%      0.1044
##      60%      0.3350
##      70%      0.5712
##      80%      0.7500
##      90%      0.8882

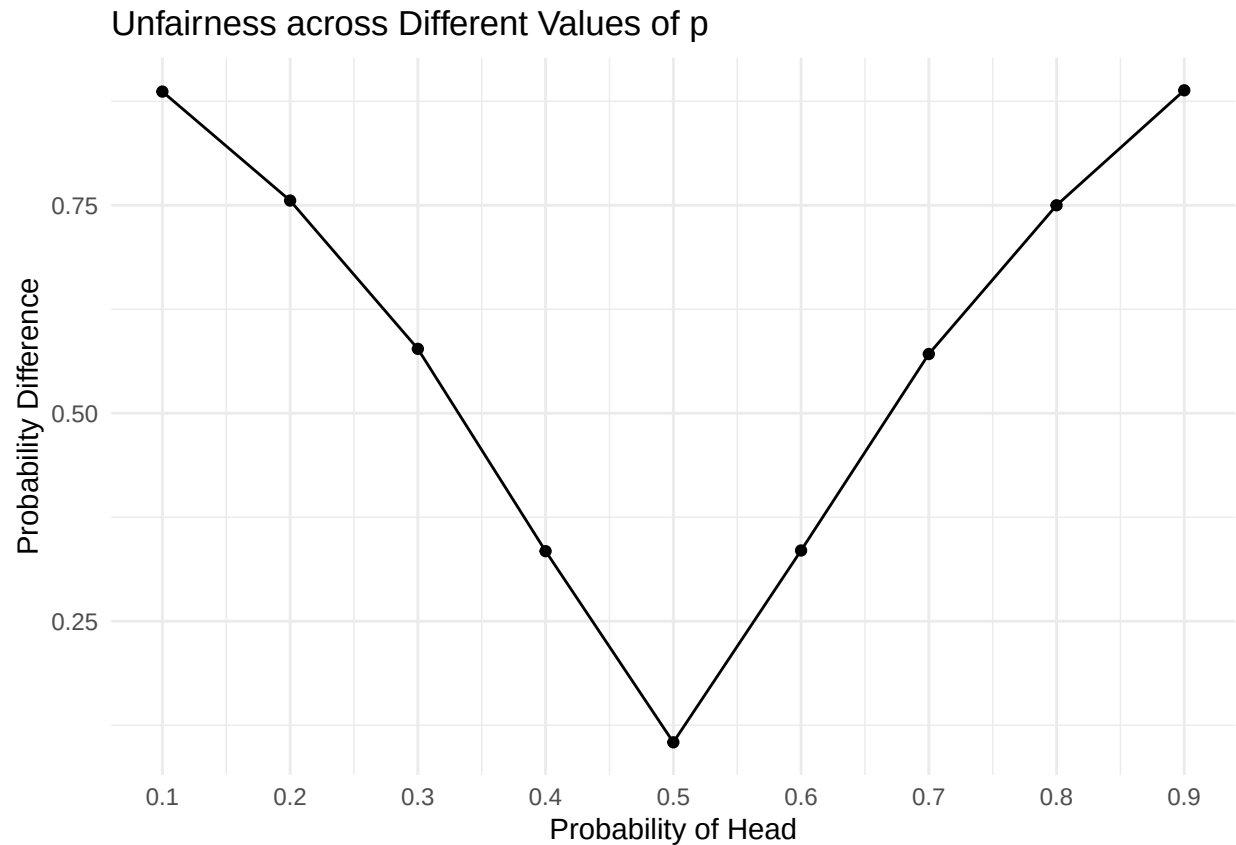
```

```

# Plot 1: p vs. unfairness
plot1_data <- data.frame(
  p = prob,
  unfairness = sapply(result, function(x) x$unfairness)
)

ggplot(
  plot1_data,
  aes(x = p, y = unfairness)
) +
  geom_line() +
  geom_point() +
  scale_x_continuous(breaks = seq(0.1, 0.9, by = 0.1)) +
  labs(
    x = "Probability of Head",
    y = "Probability Difference",
    title = "Unfairness across Different Values of p"
  ) +
  theme_minimal()

```



```
# Plot 2: Mean count for each non-host chair
plot2_data <- data.frame()

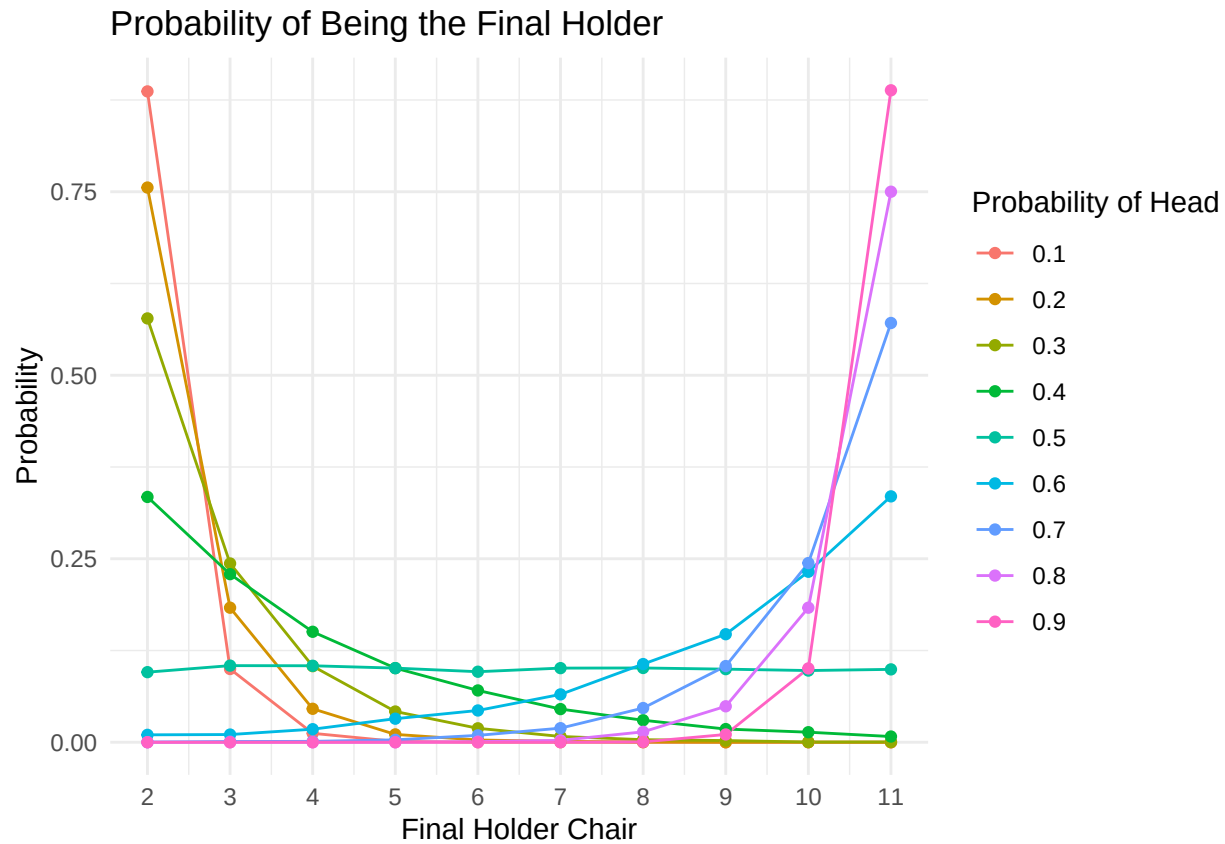
for (p in prob) {

  temp <- data.frame(
    chair = 2:chair, # exclude chair 1 as the prob is 0
    probability = result[[as.character(p)]]$probabilities[2:chair],
    p = p
  )

  plot2_data <- rbind(plot2_data, temp)
}

ggplot(
  plot2_data,
  aes(
    x = chair,
    y = probability,
    group = p,
    color = factor(p)
  )
) +
  geom_line() +
  geom_point() +
  scale_x_continuous(breaks = 2:chair) +
```

```
labs(
  x = "Final Holder Chair",
  y = "Probability",
  color = "Probability of Head",
  title = "Probability of Being the Final Holder"
) +
theme_minimal()
```



I define unfairness as the difference between the highest and lowest probabilities among the chairs of becoming the final holder. Under the fair condition ($p = 0.5$), this value should be 0 ideally. As p moves farther away from 0.5, the degree of unfairness increases. For example, when $p = 0.1$ or 0.9 , the difference between the highest and lowest probabilities is around 0.89.

Figure 1 shows that unfairness increases as p moves farther away from 0.5. Figure 2 shows the probability of each chair becoming the final holder under different values of p (nine values in total). As can be seen, when $p = 0.1$, the probability for chair 2 to be the final winner is close to 95%. From $p = 0.1$ to $p = 0.5$, the lines become increasingly flatter, indicating that the probabilities become more evenly distributed. From $p = 0.5$ to $p = 0.9$, the lines become steeper again.

Question 8 function part

```
# AI disclosure: since we are asked to use the apply fucntion, I asked chatgpt for help
# regarding how to use lapply (what we are doing is just change the outer loop to lapply)
set.seed(648)
```

```

prob <- seq(0.1, 0.9, by = 0.1)
chair <- 11
result <- list()

run_simulation <- function(p, chair = 11, n_sim = 10000) {

  final_chair <- numeric(n_sim)

  for (i in 1:n_sim) {
    final_chair[i] <- present_party(chair, p = p)[1]
  }

  counts <- table(factor(final_chair, levels = 1:chair))
  probabilities <- as.numeric(counts) / length(final_chair)

  unfairness <- max(probabilities[2:chair]) -
    min(probabilities[2:chair])

  list(
    probabilities = probabilities,
    unfairness = unfairness
  )
}

result <- lapply(prob, run_simulation, chair = chair, n_sim = 10000)

# Table 1: p and unfairness
table1 <- data.frame(
  `Prob of Head` = paste0(prob * 100, "%"),
  unfairness = sapply(result, function(x) x$unfairness),
  check.names = FALSE
)

print(table1, row.names = FALSE)

```

```

## Prob of Head unfairness
##      10%      0.8867
##      20%      0.7557
##      30%      0.5771
##      40%      0.3264
##      50%      0.0089
##      60%      0.3249
##      70%      0.5710
##      80%      0.7500
##      90%      0.8882

```

```

# Plot 1: p vs. unfairness
plot1_data <- data.frame(
  p = prob,
  unfairness = sapply(result, function(x) x$unfairness)
)

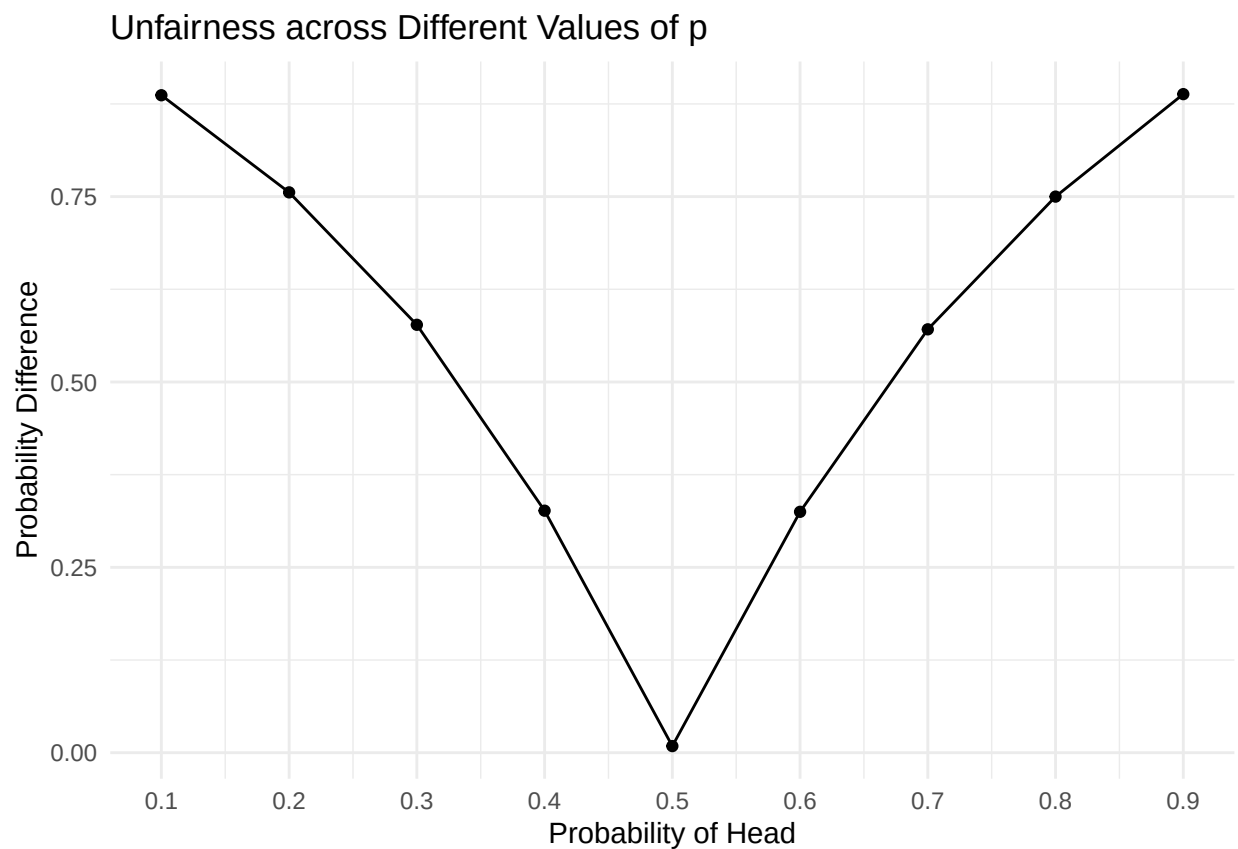
ggplot(

```

```

plot1_data,
aes(x = p, y = unfairness)
) +
geom_line() +
geom_point() +
scale_x_continuous(breaks = seq(0.1, 0.9, by = 0.1)) +
labs(
  x = "Probability of Head",
  y = "Probability Difference",
  title = "Unfairness across Different Values of p"
) +
theme_minimal()

```



```

# Plot 2: Mean count for each non-host chair
plot2_data <- data.frame()

for (j in seq_along(prob)) {

  temp <- data.frame(
    chair = 2:chair,
    probability = result[[j]]$probabilities[2:chair],
    p = prob[j]
  )

  plot2_data <- rbind(plot2_data, temp)
}

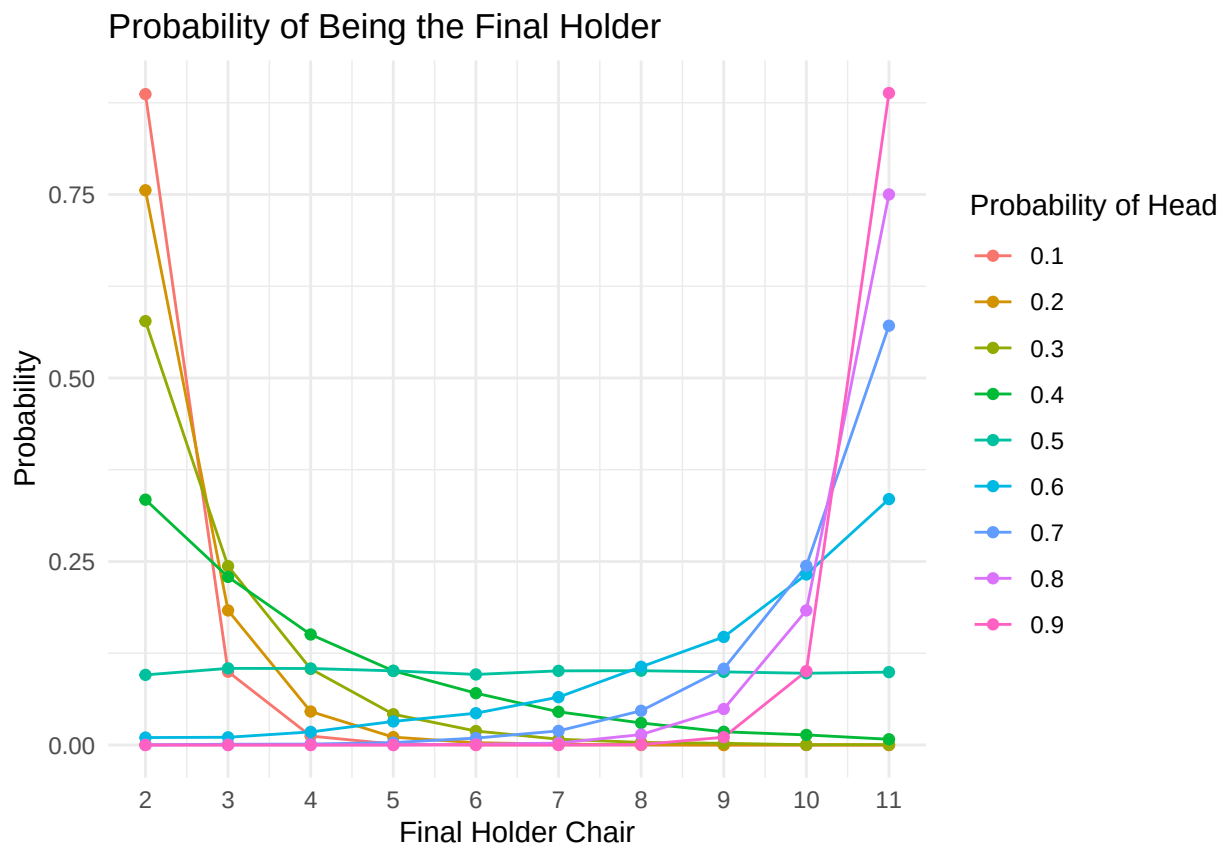
```

```

}

ggplot(
  plot2_data,
  aes(
    x = chair,
    y = probability,
    group = p,
    color = factor(p)
  )
) +
  geom_line() +
  geom_point() +
  scale_x_continuous(breaks = 2:chair) +
  labs(
    x = "Final Holder Chair",
    y = "Probability",
    color = "Probability of Head",
    title = "Probability of Being the Final Holder"
  ) +
  theme_minimal()

```



Question 9

```
set.seed(648)
prob <- seq(0.1, 0.9, by = 0.1)
chair <- 11
result <- list()

time1 <- system.time({
  for (p in prob) {

    final_chair <- numeric(100000)

    for (i in 1:100000) {
      simulation <- present_party(chair, p = p)
      final_chair[i] <- simulation[1] # final holder
    }

    # probability that each chair is the final holder
    counts <- table(factor(final_chair, levels = 1:chair))
    probabilities <- as.numeric(counts) / length(final_chair)
    # for this inner loop under given p, calculate unfairness
    unfairness <- max(probabilities) - min(probabilities)

    result[[as.character(p)]] <- list(
      probabilities = probabilities,
      unfairness = unfairness
    )
  }
})

time2 <- system.time({
  run_simulation <- function(p, chair = 11, n_sim = 100000) {

    final_chair <- numeric(n_sim)

    for (i in 1:n_sim) {
      final_chair[i] <- present_party(chair, p = p)[1]
    }

    counts <- table(factor(final_chair, levels = 1:chair))
    probabilities <- as.numeric(counts) / length(final_chair)

    unfairness <- max(probabilities[2:chair]) -
      min(probabilities[2:chair])

    list(
      probabilities = probabilities,
      unfairness = unfairness
    )
  }

  result <- lapply(prob, run_simulation, chair = chair, n_sim = 100000)
})
```

```
time1
```

```
##      user  system elapsed  
##    18.90    0.04   19.30
```

```
time2
```

```
##      user  system elapsed  
##    19.07    0.06   19.42
```

To see a larger difference in run time, I increased the number of simulations from 10000 to 100000. However, I still did not observe a significant difference between the two methods. This is reasonable because `lapply` calls a function sequentially for each element, which is essentially equivalent to a for loop.

Question 10

I would choose the function approach because it can be easily reused. Changing an input can affect multiple places at the same time, and the longer the code is, the greater the benefit becomes. Since the code we wrote today was quite short, the difference is not very significant. In my future vibe coding, I will try to have Codex write functions whenever possible, and the more parameters, the better.

Question 11

Whether a result is considered fair is somewhat subjective. Under my definition, I consider a probability difference of 0.1 to already be unfair, so only $p = 0.5$ would be considered fair. I interpret the question as asking how unfairness changes as p moves away from 0.5: whether it increases at a decreasing rate, linearly, or at an increasing rate. If the increase were slow at first, there could be slightly biased values of p for which the level of unfairness remains relatively small. However, Plot 1 shows that unfairness increases approximately linearly as p moves away from 0.5. Therefore, I would conclude that the fair result is fragile rather than robust to bias in the coin.

Question 12

I originally planned to add another loop to my current code and vary the number of chairs from 5 to 21. However, I realized that my current definition of fairness depends on the number of chairs, so directly comparing unfairness across different table sizes would not be appropriate. I would need to develop a different fairness measure that allows for meaningful comparisons across different numbers of chairs. For now, I will leave this question as it is.